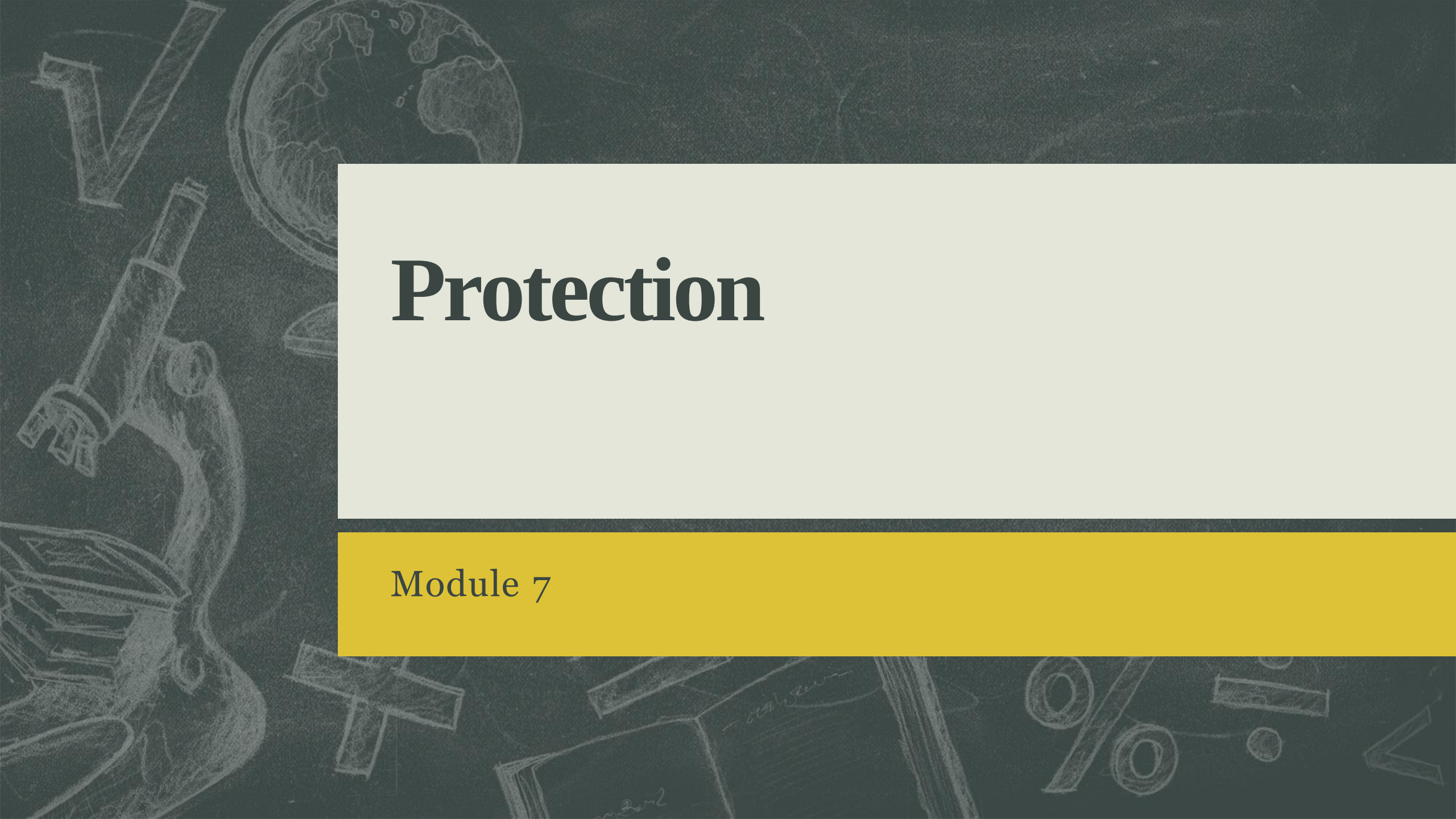




Protection

Module 7

System Protection

- Protection **tackles** the system's **internal threats**.
- A mechanism for **controlling access** to **processes**, **programs**, and user **resources**
- It specifies **which files** a specific **user can access or view and modify** to maintain the proper functioning of the system.

System Protection - Components

▪ Access Control:

- Determines who can access what resources and under what conditions.
- Enforced through authentication, authorization, and auditing mechanisms.

▪ Isolation:

- Ensures that processes and resources are separated to prevent interference and unauthorized access.
- Implemented through memory protection, virtualization, and sandboxing techniques.

System Protection - Components

- Resource Management:

- Manages the allocation and usage of system resources such as CPU, memory, and I/O devices.
- Enforces fairness and prevents resource starvation through scheduling algorithms and quotas.

- Authentication:

- Verifies the identity of users and processes before granting access to resources.
- Uses credentials such as passwords, digital certificates, or biometric data for authentication.

System Protection - Components

■ Encryption:

- Protects data from unauthorized access or interception by encrypting it using cryptographic algorithms.
- Ensures confidentiality and integrity of data during storage, transmission, and processing.

■ Auditing:

- Monitors and logs system activities to track user actions, detect security violations, and investigate incidents.
- Provides accountability and helps in forensic analysis and compliance auditing.

System Protection – Access Matrix

System Protection – Access Matrix

- Used to **define** each **process's rights** for **each object** executing in the domain.
- The access matrix is represented as a two-dimensional matrix.
- A process may operate within a protection domain, which specifies the resources that the process may access.
- Each domain defines a **set of objects and the types of operations** that may be invoked on each object.
- The ability to execute an operation on an object is an **access right**.

System Protection – Access Matrix

- The **columns** of the matrix **represent** **objects**, while the **rows** represent **domains**.
- Each matrix cell represents a specific set of access rights that are granted to processes of the domain this indicates that each entry access (i, j) specifies the set of actions that a process executing in domain D_i may invoke on object O_j .

System Protection – Access Matrix

- Rows represent subjects (e.g., users, processes).
- Columns represent objects (e.g., files, devices).
- Entries in the matrix specify the access rights or permissions that each subject has on each object

System Protection – Access Matrix - Structure

- Each cell in the matrix holds a set of permissions, defining what actions a particular subject can perform on a specific object. Common permissions include:
 - Read – permission to read the contents of the object.
 - Write – permission to modify the contents of the object.
 - Execute – permission to execute or run the object (applicable to files or programs).
 - Delete – permission to delete the object.

System Protection – Access Matrix

- It occupies a significant amount of memory and is very sparse.
- Four widely used access matrix implementations can be formed using these decomposition methods:
 - Global Table
 - Capability Lists for Domains
 - Access Lists for Objects
 - Lock and key Method

System Protection – Access Matrix – Global Table

- Most basic and simple implementation
- A set of an ordered triple **<domain , object, right-set>**. When an operation M is being executed on an object O_j within domain D_i
- If the triple is present, the operation can proceed to continue, or else a condition of an exception is thrown.
- The table is sometimes too large, it cannot be stored in the main memory, that's why input and output are required additionally.

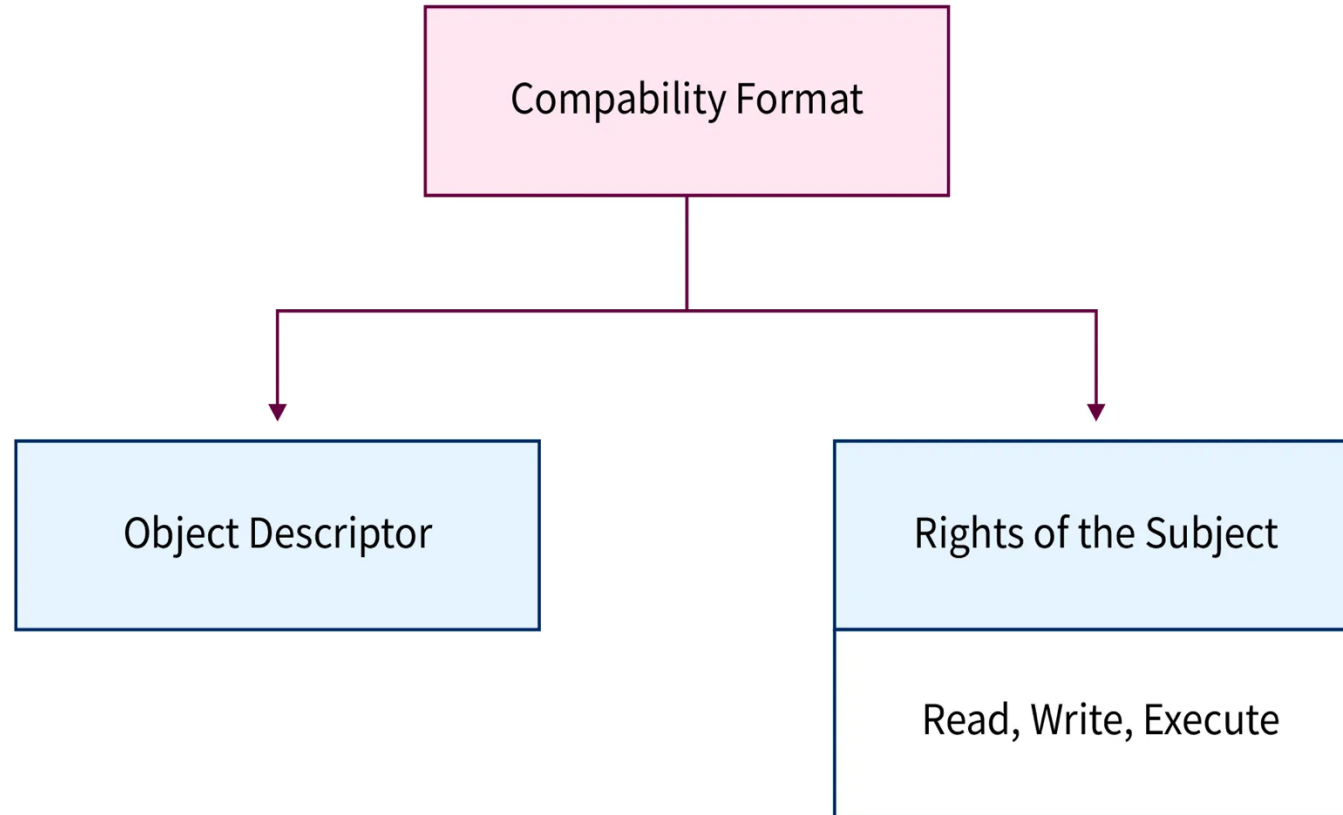
System Protection – Access Matrix – Access Lists

- When an operation M is being executed on an object O_j within domain D_i , We search for an entry $\langle \text{Domain}(D_i), \text{right-set}(R_k) \rangle$ with M belongs R_k in the access list for object O_j .
- If the triple is present, the operation can proceed to continue, or else we check the initial set. If M is included in the default set, access is allowed; otherwise, access is denied, and an exception is raised.

System Protection – Access Matrix – Capability Lists

- Capability Lists is a collection of objects and the operations that can be performed on them.
- The object here is specified by a physical name called capability.
- In this method, we can associate each row with its domain instead of connecting the columns of the access matrix to the objects as an access list.
- A capability list is itself a protected object maintained by OS and accessed by the user indirectly.

System Protection – Access Matrix – Capability Lists



System Protection – Access Matrix – Lock Key

- Every **domain** has a distinct **bit pattern** called **keys**, and every **object** has a **distinct bit pattern** called **locks**.
- Only if a domain's key matches one of the locks of the object, A process can access it.
- When a process running in a specific domain (D_i) try to access an object (O_j) then the key to that D_i must match with the lock of that O_j , then only an object can be accessed.

System Protection – Access Matrix – Example

- When there are four files having the following access rights (files f1, f2, f3, and f4) and three domains (D1, D2, D3)

ACCESS MATRIX				
Object Domain	File 1	File 2	File 3	File 4
D1	<i>read</i> <i>write</i>	<i>read</i>	<i>read</i>	<i>read</i>
D2	<i>read</i>	<i>write</i>	-	<i>exceute</i>
D3	<i>write</i>	-	<i>write</i>	<i>exceute</i>

System Protection – Access Matrix – Example

- Any file can be read by a process running in the D1 domain and can only be written into f1.
- A process in D2 domain has access to read f1, write to f2, and execute f4.
- D3 processes in the domain can write to f1 and 3 and can execute f4.
- The contents of the access-matrix entries are normally decided by the users.

System Protection – Access Matrix – Example

- An access matrix can be used to specify and implement both static and dynamic access rights.
- When a process switches between different domains, it's like when we do a (domain switch) operation on an object (the domain).

System Protection – Access Matrix – Example

ACCESS MATRIX WITH DOMAIN AS OBJECTS						
Object Domain	File 1	File 2	File 3	D1	D2	D3
D1	<i>read</i> <i>write</i>	<i>read</i>	<i>write</i>	-	<i>switch</i>	-
D2	<i>write</i>	<i>exceute</i>	<i>write</i>	<i>switch</i>	-	<i>switch</i>
D3	<i>read</i>	-	<i>exceute</i>	-	-	-

System Protection – Access Matrix – Example - Simple

domain \ object	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

System Protection – Access Matrix – Example - Domains

domain \ object	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

System Protection – Access Matrix – Copy – Limited Copy

object domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute		

(a)

object domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute	read	

(b)

System Protection – Access Matrix – Example – Owner Rights

object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		read* owner	read* owner write
D_3	execute		

(a)

object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		owner read* write*	read* owner write
D_3		write	write

(b)

Capability Based System

Why Capability Based System?

- Capability-based systems address the issues raised above of OS security.
- Issues related to information flow barriers between different users/processes should be addressed.
- Resource access and sharing must be tightly controlled according to specific, and restricting policies with a variety of desirable properties.
- **Capability-based systems** address the issues around effectively harnessing system resources. They enable the **safe and controlled access to resources that are very low-level.**

Capability Based System

- OS that uses **capabilities** as a primary mechanism for **controlling access** to **memory** and **system resources**. Capabilities are **keys** that allow users to access resources and perform operations on them
- A capability is a **communicable, unforgeable token of authority**, defining **what actions** (or operations) the holder of the capability can **perform on an object** (e.g., file, process, device) in the system.
- Capabilities allow an OS **to enforce access control** in a **fine-grained and flexible manner**, ensuring that only authorized entities can access specific system resources.

Capability Based System

- A **capability** is a kind of **secure token** that represents **both the reference** to a specific **system resource** (like a file or device) and the **allowed actions** a **user or process** can take on that resource (like read, write, or execute).
- In traditional systems, access control often involves a **central list** (like Access Control Lists, or ACLs) that specifies what each user or process is allowed to do with each resource.
- While a capability-based system **distributes access control through capabilities**, which are **given directly** to the processes or users that need them.

Capability Based System

- The concept of a capability-based system is simple. Each user-level component accesses resources through a level of indirection. **Resource accesses are made by providing a key**, the possession of which denotes the ability to perform a set of operations on a resource. **Keys can be passed** (delegated) **between components** to enable the sharing of resources.
- For example, the ability to communicate with another component is only allowed through the possession of a key, and if a component passes that key to another, then they both can access the resource through it. Capabilities are these keys.

Capability Based System

Each capability acts as both:

- A reference to a resource.
 - A set of permissions for actions on that resource
-
- Capabilities are **stored in capability lists** (C-lists) associated with each process or user.
 - Each capability in the list corresponds to a resource and defines what that process can do with it.

Capability Based System

- When a process/User tries to access a resource:
 - The OS checks if the process's **C-list** contains a capability for that resource.
 - If it does, the OS verifies if the capability allows the requested action (e.g., read, write).
 - If the capability permits the action, the OS grants access; otherwise, it denies access.

Capability Based System – Hydra as Example

- Hydra is a **capability-based protection system** that provides considerable flexibility.
- The system implements a **fixed set of possible access rights**, including such basic forms of access as the right to read, write, or execute a memory segment.
- It includes both **system-defined rights** and **user-defined rights**.

Capability Based System – Hydra as Example

- The interpretation of **user-defined rights** is up to the **specific user programs**, but the **OS provides support for protecting access to those rights**, whatever they may be.
- **Operations on objects** are defined procedurally, and those **procedures** are **themselves protected objects**, accessed indirectly through capabilities.
- When **users define new procedures** (i.e., custom operations or user-defined procedures), these procedures must be **registered with Hydra's protection system**. By identifying these custom operations to the protection system, Hydra can manage access to them alongside system-defined rights.

Capability Based System – Hydra as Example

- When an object is created, the names of operations defined on that object become **auxiliary rights**, described in a capability for an instance of the type.
- For a process to act on an object, the capabilities it holds for that object must contain the name of the operation being invoked among its auxiliary rights.
- This **allows access to be controlled on an instance-by-instance and process-by-process basis.**

Capability Based System – Hydra as Example

- Hydra also allows **rights amplification**, in which a process is deemed to be trustworthy, and thereby allowed to act on any object corresponding to its parameters.
- Objects can be moved from amplified state to unamplified state. The dynamic adjustment of rights is performed **to guarantee consistency** of a programmer-defined abstraction.
- When a user passes an object as an argument to a procedure, we may need to ensure that the procedure cannot modify the object. However, if amplification may occur, the right to modify .may be reinstated.

Capability Based System – Hydra as Example

- A user may trust that a procedure performs its task correctly. This assumption is not always correct, however, because of hardware or software errors. **Hydra solves this problem by restricting amplifications.**
- The procedure-call mechanism of Hydra was designed as a direct solution to the problem of mutually suspicious subsystems.
- A Hydra subsystem is built on top of its protection kernel and may require protection of its own components.
- A subsystem interacts with the kernel through calls on a set of kernel-defined primitives that define access rights to resources defined by the subsystem. The subsystem designer can define policies for use of these resources by user processes.